

# 数据库实验报告（第九周）

班级：3班

姓名：梁力航

学号：23336128

## 一、实验目的

- 理解实体完整性约束的概念和作用
- 掌握主键约束的创建和使用方法
- 理解主键约束对数据插入、更新操作的影响
- 掌握事务处理中主键冲突的处理机制
- 理解XACT\_ABORT设置对事务回滚的影响

## 二、实验内容

- 在school数据库中建立一张新表class，包括class\_id(varchar(4)), name(varchar(10)), department(varchar(20))三个列，并约束class\_id为主键。
- 创建事务T3，在事务中插入一个元组 ('0001', '01CSC','CS')，并在T3中嵌套创建事务T4，T4也插入和T3一样的元组，编写代码测试，查看结果。
- 在表class中，尝试设置name='01CSC'的记录的class\_id为NULL，查看结果。
- 在表class中，不创建事务，插入两个元组 ('0002', '01CSC','CS')，('0002', '03CSC', 'CS')，然后查看表中几条记录，为什么？
- 在表class中，创建事务，并设置开启回滚，然后插入两个元组 ('0003', '03CSC','CS')，('0001', '03CSC', 'CS')，查看结果，表中几条记录？
- 在完成上面几步的前提下，尝试设置'name'为主键，看能否成功，并思考原因。

说明：本实验在SQL Server环境下进行，使用School\_Data数据库。

## 三、实验结果

### (1) 创建class表并设置主键

SQL代码：

```
CREATE TABLE class(  
    class_id varchar(5) PRIMARY KEY,  
    name varchar(10),  
    department varchar(20)  
);
```

验证表结构：

```
EXEC sp_help 'class';
```

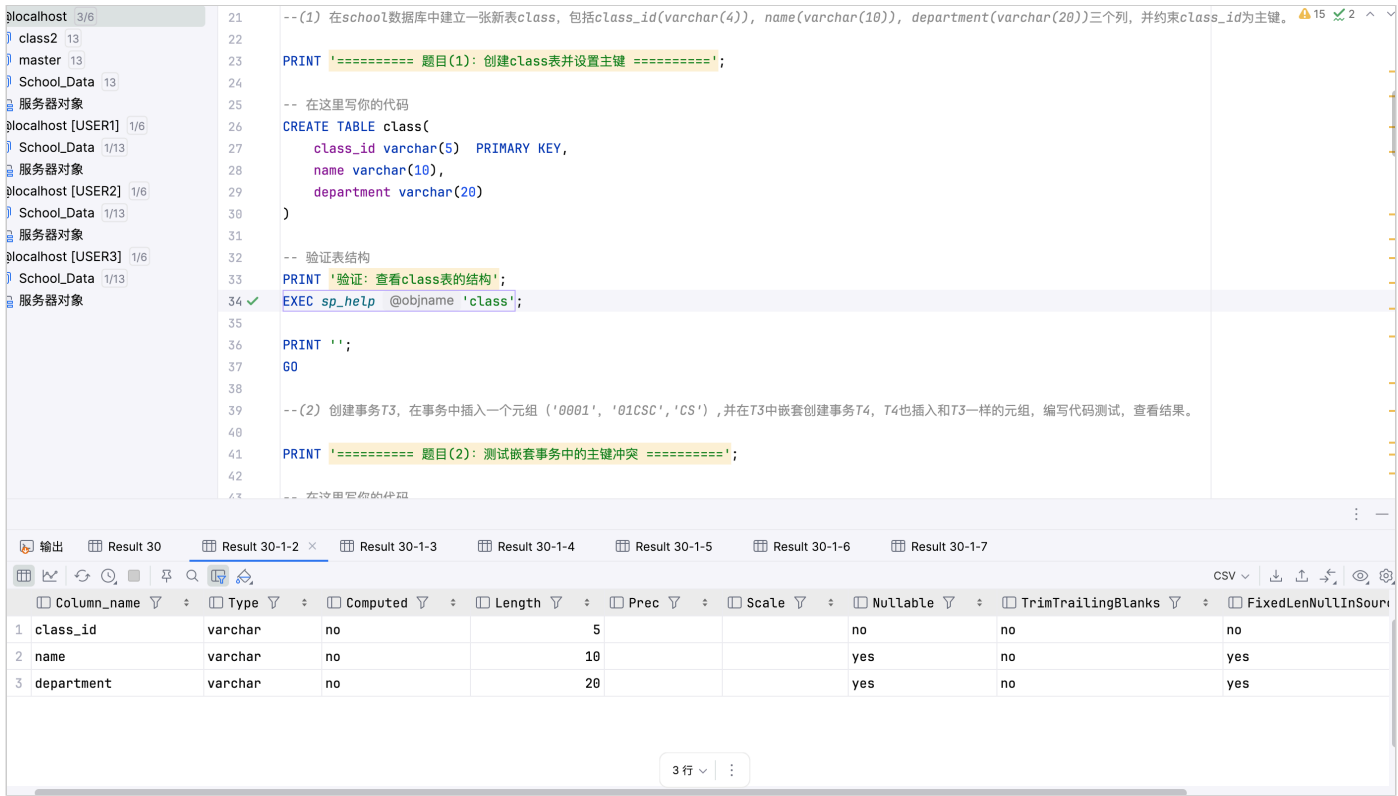


图1：查看class表结构

成功创建了class表，class\_id列被设置为主键。从表结构可以看到，主键约束自动包含了NOT NULL约束和UNIQUE约束。

终端输出：

```
===== 题目(1)：创建class表并设置主键 =====
验证：查看class表的结构
[2025-11-06 11:04:01] 在 2 s 518 ms (execution: 55 ms, fetching: 2 s 463 ms) 内检索到从 1 开始的 1 行
```

(2) 测试嵌套事务中的主键冲突

SQL代码：

```
SET XACT_ABORT OFF

BEGIN TRANSACTION T3
    INSERT INTO class VALUES ('0001','01CSC','CS');

    BEGIN TRANSACTION T4
        INSERT INTO class VALUES ('0001','01CSC','CS'); -- 违反主键约束
    COMMIT TRANSACTION T4

COMMIT TRANSACTION T3
```

```
50
51 SET XACT_ABORT OFF
52 BEGIN TRANSACTION T3
53 insert into class values ( class_id '0001', name '01CSC', department 'CS');
54 BEGIN Transaction T4
55 insert into class values ( class_id '0001', name '01CSC', department 'CS');
56 COMMIT TRANSACTION T4
57 COMMIT TRANSACTION T3
58
59
60 -- 查看表中的数据
61 PRINT '查看表中的数据: ';
```

[23000][2627] 行 5: Violation of PRIMARY KEY constraint 'PK\_class\_FDF4798634627C12'. Cannot insert duplicate key in object 'dbo.class'. The duplicate key value is (0001).

图2：嵌套事务中的主键冲突报错

@localhost 3/6

class2 13

master 13

School\_Data 13

服务器对象

@localhost [USER1] 1/6

School\_Data 1/13

服务器对象

@localhost [USER2] 1/6

School\_Data 1/13

服务器对象

@localhost [USER3] 1/6

School\_Data 1/13

服务器对象

```
49 -- 5. 观察是否会报错
50
51 SET XACT_ABORT OFF
52 BEGIN TRANSACTION T3
53 insert into class values ( class_id '0001', name '01CSC', department 'CS');
54 BEGIN Transaction T4
55 insert into class values ( class_id '0001', name '01CSC', department 'CS');
56 COMMIT TRANSACTION T4
57 COMMIT TRANSACTION T3
58
59
60 -- 查看表中的数据
61 PRINT '查看表中的数据: ';
```

62 ✓ SELECT \* FROM class;

63

64 PRINT '';

65 GO

66

67 -- (3) 在表class中, 尝试设置name='01CSC'的记录class\_id 为NULL, 查看结果

68

69 PRINT '===== 题目(3): 尝试将主键设置为NULL =====';

70

输出 Result 32 Result 30-1-2 Result 30-1-3 Result 30-1-4 Result 30-1-5 Result 30-1-6 Result 30-1-7

| class_id | name  | department |
|----------|-------|------------|
| 1 0001   | 01CSC | CS         |

1行

图3：事务结束后查看class表内容

终端输出：

```
===== 题目(2)：测试嵌套事务中的主键冲突 =====
[2025-11-06 11:04:01] [S0000][3621] The statement has been terminated.
查看表中的数据:
[2025-11-06 11:04:01] 19 ms 中有 1 行受到影响
[2025-11-06 11:04:01] [23000][2627] 行 15: Violation of PRIMARY KEY constraint 'PK__class__FDF4798636B93626'.
Cannot insert duplicate key in object 'dbo.class'. The duplicate key value is (0001).
```

实验结果分析：

- T3中的第一条INSERT成功执行，插入了('0001','01CSC','CS')
- T4中的第二条INSERT违反主键约束，报错：Violation of PRIMARY KEY constraint

- 由于设置了XACT\_ABORT OFF，T4的错误不会导致T3自动回滚
- T3成功提交，表中有1条记录
- 这说明在XACT\_ABORT OFF模式下，嵌套事务中的错误不会影响外层事务的提交

### (3) 尝试将主键设置为NULL

SQL代码：

```
UPDATE class SET class_id = NULL WHERE name = '01CSC';
```

```
66
67 --(3) 在表class中，尝试设置name='01CSC'的记录class_id 为NULL，查看结果
68
69 PRINT '===== 题目(3)：尝试将主键设置为NULL =====';
70
71 -- 在这里写你的代码
72 UPDATE class SET class_id = NULL where name = '01CSC';
73
74 -- 查看表中的数据
75 PRINT '查看表中的数据: ';
76 SELECT * FROM class;
77
78 PRINT '';
79 GO
80
81 --(4) 在表class中，不创建事务，插入两个元组 ('0002', '01CSC', 'CS')， ('0002', '03CSC', 'CS')，然后查看表中有几条记录，为什么
--
[23000][515] 行 1: Cannot insert the value NULL into column 'class_id', table 'School_Data.dbo.class'; column does not allow nulls. UPDATE fails.
```

图4：尝试将主键设置为NULL时报错

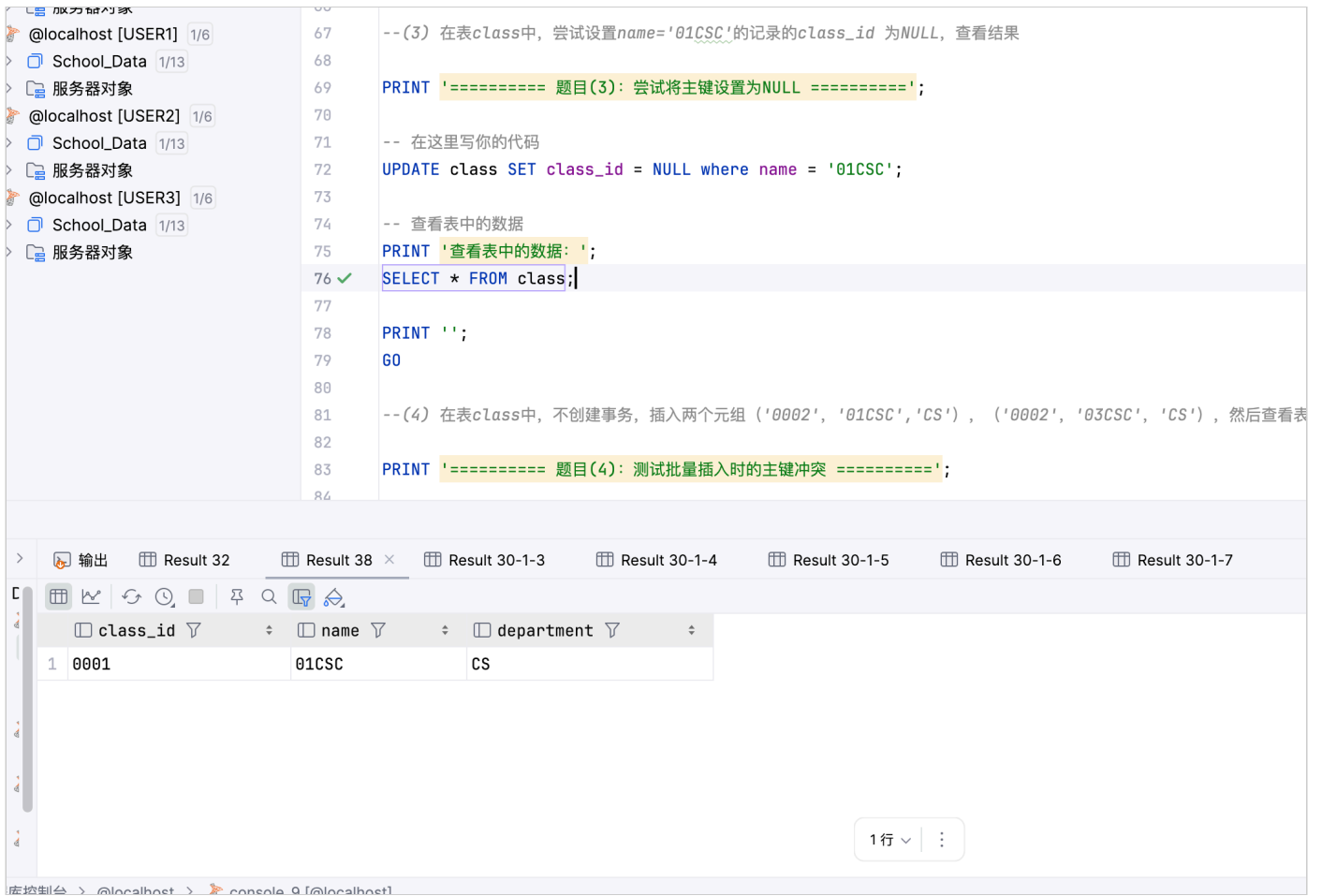


图5：验证主键值未被修改为NULL

#### 终端输出：

```
===== 题目(3): 尝试将主键设置为NULL =====
[2025-11-06 11:04:03] [23000][515] 行 4: Cannot insert the value NULL into column 'class_id',
table 'School_Data.dbo.class'; column does not allow nulls. UPDATE fails.
[2025-11-06 11:04:03] [50000][3621] The statement has been terminated.
查看表中的数据:
```

#### 实验结果分析：

- 报错信息：Cannot insert the value NULL into column 'class\_id'
- 主键约束包含两个限制：NOT NULL约束和UNIQUE约束
- 主键列不允许为NULL，这是实体完整性的基本要求
- UPDATE操作失败，表中数据保持不变

#### (4) 测试批量插入时的主键冲突

##### SQL代码：

```
INSERT INTO class VALUES ('0002','01CSC','CS');
INSERT INTO class VALUES ('0002','03CSC','CS');
```

```
80
81 --(4) 在表class中, 不创建事务, 插入两个元组 ('0002', '01CSC', 'CS'), ('0002', '03CSC', 'CS'), 然后查看表中几条记录, 为什么?
82
83 PRINT '===== 题目(4): 测试批量插入时的主键冲突 =====';
84
85 -- 在这里写你的代码
86 INSERT INTO class values ( class_id '0002', name '01CSC', department 'CS');
87 INSERT INTO class values ( class_id '0002', name '03CSC', department 'CS');
88
89 -- 查看表中的数据
90
91 PRINT '查看表中有几条记录: ';
92 SELECT COUNT(*) AS 记录数 FROM class;
93 SELECT * FROM class;
94
95 PRINT '';
```

[23000][2627] 行 2: Violation of PRIMARY KEY constraint 'PK\_\_class\_\_FDF4798634627C12'. Cannot insert duplicate key in object 'dbo.class'. The duplicate key value is (0002).

图6：第二条INSERT语句违反主键约束

localhost [USER1] 1/6  
School\_Data 1/13  
服务器对象  
localhost [USER2] 1/6  
School\_Data 1/13  
服务器对象  
localhost [USER3] 1/6  
School\_Data 1/13  
服务器对象

```
81 --(4) 在表class中, 不创建事务, 插入两个元组 ('0002', '01CSC', 'CS'), ('0002', '03CSC', 'CS'), 然后查看表中几条记录, 为什么?
82
83 PRINT '===== 题目(4): 测试批量插入时的主键冲突 =====';
84
85 -- 在这里写你的代码
86 INSERT INTO class values ( class_id '0002', name '01CSC', department 'CS');
87 INSERT INTO class values ( class_id '0002', name '03CSC', department 'CS');
88
89 -- 查看表中的数据
90
91 PRINT '查看表中有几条记录: ';
92 SELECT COUNT(*) AS 记录数 FROM class;
93 SELECT * FROM class;
94
95 PRINT '';
```

输出 Result 32 Result 38 Result 40 × Result 30-1-4 Result 30-1-5 Result 30-1-6 Result 30-1-7

| class_id | name  | department |
|----------|-------|------------|
| 1 0001   | 01CSC | CS         |
| 2 0002   | 01CSC | CS         |

2行

图7：表中只有第一条INSERT成功

终端输出：

```
===== 题目(4): 测试批量插入时的主键冲突 =====
[2025-11-06 11:04:03] [S0000][3621] The statement has been terminated.
查看表中有几条记录:
[2025-11-06 11:04:03] 11 ms 中有 1 行受到影响
[2025-11-06 11:04:03] [23000][2627] 行 5: Violation of PRIMARY KEY constraint 'PK__class__FDF4798636B93626'.
Cannot insert duplicate key in object 'dbo.class'. The duplicate key value is (0002).
```

实验结果分析：

- 第一条INSERT成功执行，插入了('0002','01CSC','CS')
- 第二条INSERT违反主键约束，报错：Violation of PRIMARY KEY constraint
- 表中有2条记录：('0001','01CSC','CS')和('0002','01CSC','CS')
- **原因：**在SQL Server中，每条INSERT语句是一个独立的语句。第一条INSERT成功后立即提交（因为没有显式事务），第二条INSERT失败不会影响第一条已提交的记录

- 这与在事务中的行为不同，如果在事务中，可以通过ROLLBACK撤销所有操作

## (5) 测试事务回滚对主键冲突的影响

### 实验5.1：使用XACT\_ABORT ON

```
SET XACT_ABORT ON

BEGIN TRANSACTION T
    INSERT INTO class VALUES ('0003','03CSC','CS');
    INSERT INTO class VALUES ('0001','03CSC','CS'); -- 违反主键约束
COMMIT TRANSACTION T
```

```
98 -- (5) 在表class中，创建事务，并设置开启回滚，然后插入两个元组 ('0003', '03CSC', 'CS'), ('0001', '03CSC', 'CS'), 查看结果，表中几条记录?
99
100 PRINT '===== 题目(5): 测试事务回滚对主键冲突的影响 =====';
101
102 -- 在这里写你的代码
103 -- 提示:
104 -- 1. 使用BEGIN TRANSACTION创建事务
105 -- 2. 插入第一条记录 ('0003', '03CSC', 'CS')
106 -- 3. 插入第二条记录 ('0001', '03CSC', 'CS') - 这条会冲突
107 -- 4. 使用ROLLBACK回滚事务
108 SET XACT_ABORT ON
109 BEGIN TRANSACTION T
110 INSERT INTO class values ( class_id '0003', name '03CSC', department 'CS');
111 INSERT INTO class values ( class_id '0001', name '03CSC', department 'CS');
112 Commit TRANSACTION T
113
114 -- 查看表中有几条记录
115 PRINT '查看表中有几条记录: ';
116 SELECT COUNT(*) AS 记录数 FROM class;
117 SELECT * FROM class;
118
```

[23000][2627] 行 4: Violation of PRIMARY KEY constraint 'PK\_class\_FDF4798634627C12'. Cannot insert duplicate key in object 'dbo.class'. The duplicate key value is (0001).

图8: XACT\_ABORT ON时主键冲突报错

服务器对象  
calhost [USER1] 1/6  
School\_Data 1/13  
服务器对象  
calhost [USER2] 1/6  
School\_Data 1/13  
服务器对象  
calhost [USER3] 1/6  
School\_Data 1/13  
服务器对象

106 -- 3. 插入第二条记录 ('0001', '03CSC', 'CS') - 这条会冲突  
107 -- 4. 使用ROLLBACK回滚事务  
108 SET XACT\_ABORT ON  
109 BEGIN TRANSACTION T  
110 INSERT INTO class values ( class\_id '0003', name '03CSC', department 'CS');  
111 INSERT INTO class values ( class\_id '0001', name '03CSC', department 'CS');  
112 Commit TRANSACTION T  
113  
114 -- 查看表中几条记录  
115 PRINT '查看表中有几条记录: ';  
116 SELECT COUNT(\*) AS 记录数 FROM class;  
117 ✓ SELECT \* FROM class;  
118  
119 PRINT '';  
120 GO  
121  
122 -- (6) 在完成上面几步的前提下, 尝试设置'name'为主键, 看能否成功, 并思考原因。  
123  
124 PRINT '===== 题目(6): 尝试将name列设置为主键 =====';

输出 Result 32 Result 38 Result 40 Result 42 × Result 30-1-5 Result 30-1-6 Result 30-1-7

class\_id name department

000101CSCCS

000201CSCCS

2行

图9: XACT\_ABORT ON模式下事务自动回滚, 表中仍为2条记录

终端输出:

```
[2025-11-06 10:59:45] 21 ms 中有 1 行受到影响
[2025-11-06 10:59:45] [23000][2627] 行 4: Violation of PRIMARY KEY constraint 'PK__class__FDF4798634627C12'.
Cannot insert duplicate key in object 'dbo.class'. The duplicate key value is (0001).
[2025-11-06 11:00:18] 在 349 ms (execution: 4 ms, fetching: 345 ms) 内检索到从 1 开始的 2 行
```

XACT\_ABORT ON模式结果:

- 第一条INSERT成功执行
- 第二条INSERT违反主键约束, 报错
- 由于XACT\_ABORT ON, 错误自动回滚整个事务
- 第一条INSERT也被回滚, 表中仍然只有2条记录

实验5.2: 使用XACT\_ABORT OFF

```
SET XACT_ABORT OFF

BEGIN TRANSACTION T
    INSERT INTO class VALUES ('0003','03CSC','CS');
    INSERT INTO class VALUES ('0001','03CSC','CS'); -- 违反主键约束
COMMIT TRANSACTION T
```



```
118
119 SET XACT_ABORT OFF
120 BEGIN TRANSACTION T
121 INSERT INTO class values ( class_id '0003', name '03CSC', department 'CS');
122 INSERT INTO class values ( class_id '0001', name '03CSC', department 'CS');
123 Commit TRANSACTION T
124
125 -- 查看表中有几条记录
126 PRINT '查看表中有几条记录: ';
127 SELECT COUNT(*) AS 记录数 FROM class;
128 SELECT * FROM class;
129
130 PRINT '';
```

[23000][2627] 行 4: Violation of PRIMARY KEY constraint 'PK\_\_class\_\_FDF4798634627C12'. Cannot insert duplicate key in object 'dbo.class'. The duplicate key value is (0001). ×

图10: XACT\_ABORT OFF时主键冲突报错

localhost 3/6  
class2 13  
master 13  
School\_Data 13  
服务器对象  
localhost [USER1] 1/6  
School\_Data 1/13  
服务器对象  
localhost [USER2] 1/6  
School\_Data 1/13  
服务器对象  
localhost [USER3] 1/6  
School\_Data 1/13  
服务器对象

```
110 INSERT INTO class values ( class_id '0003', name '03CSC', department 'CS');
111 INSERT INTO class values ( class_id '0001', name '03CSC', department 'CS');
112 Commit TRANSACTION T
113
114 -- 查看表中有几条记录
115 PRINT '查看表中有几条记录: ';
116 SELECT COUNT(*) AS 记录数 FROM class;
117 SELECT * FROM class;
118
119 SET XACT_ABORT OFF
120 BEGIN TRANSACTION T
121 INSERT INTO class values ( class_id '0003', name '03CSC', department 'CS');
122 INSERT INTO class values ( class_id '0001', name '03CSC', department 'CS');
123 Commit TRANSACTION T
124
125 -- 查看表中有几条记录
126 PRINT '查看表中有几条记录: ';
127 SELECT COUNT(*) AS 记录数 FROM class;
128 ✓ SELECT * FROM class;
129
130 PRINT '';
131 GO
```

输出 Result 32 Result 38 Result 40 Result 42 Result 44 × Result 30-1-6 Result 30-1-7 CSV

|   | class_id | name  | department |
|---|----------|-------|------------|
| 1 | 0001     | 01CSC | CS         |
| 2 | 0002     | 01CSC | CS         |
| 3 | 0003     | 03CSC | CS         |

3行

图11: XACT\_ABORT OFF模式下第一条INSERT被提交，表中有3条记录

终端输出：

```
[2025-11-06 11:00:49] [S0000][3621] The statement has been terminated.
[2025-11-06 11:00:49] 14 ms 中有 1 行受到影响
[2025-11-06 11:00:49] [23000][2627] 行 4: Violation of PRIMARY KEY constraint 'PK__class__FDF4798634627C12'.
Cannot insert duplicate key in object 'dbo.class'. The duplicate key value is (0001).
[2025-11-06 11:01:08] 在 355 ms (execution: 4 ms, fetching: 351 ms) 内检索到从 1 开始的 3 行
```

XACT\_ABORT OFF模式结果：

- 第一条INSERT成功执行
- 第二条INSERT违反主键约束，报错
- 由于XACT\_ABORT OFF，错误不会自动回滚事务
- COMMIT语句仍然执行，提交了第一条成功的INSERT
- 表中有3条记录：('0001',...), ('0002',...), ('0003',...)

## 对比总结：

XACT\_ABORT设置决定了事务中发生错误时的行为。ON模式下，任何运行时错误都会自动回滚整个事务，保证了事务的原子性。OFF模式下，错误只会终止当前语句，事务可以继续执行或提交，这给了开发者更多的控制权，但也需要更谨慎地处理错误。

### (6) 尝试将name列设置为主键

SQL代码：

```
ALTER TABLE class ADD CONSTRAINT class_name_pk PRIMARY KEY (name);
```

```
133 --(6) 在完成上面几步的前提下，尝试设置'name'为主键，看能否成功，并思考原因。
134
135 PRINT '===== 题目(6)：尝试将name列设置为主键 =====';
136
137 -- 在这里写你的代码
138 -- 提示：
139 -- 1. 先删除原有的主键约束
140 -- 2. 尝试在name列上创建主键约束
141 ALTER TABLE class ADD CONSTRAINT class_name_pk PRIMARY KEY (name);
142
143 -- 查看表结构
144 PRINT '查看表结构: ';
145 EXEC sp_help @objname 'class';
146
147 -- 查看表中的数据
148 PRINT '查看表中的数据: ';
149 SELECT * FROM class;
150
151 PRINT '';
```

```
[S0000][1779] 行 1: Table 'class' already has a primary key defined on it.
[S0000][1750] 行 1: Could not create constraint or index. See previous errors.
```

图12：尝试添加第二个主键约束时报错

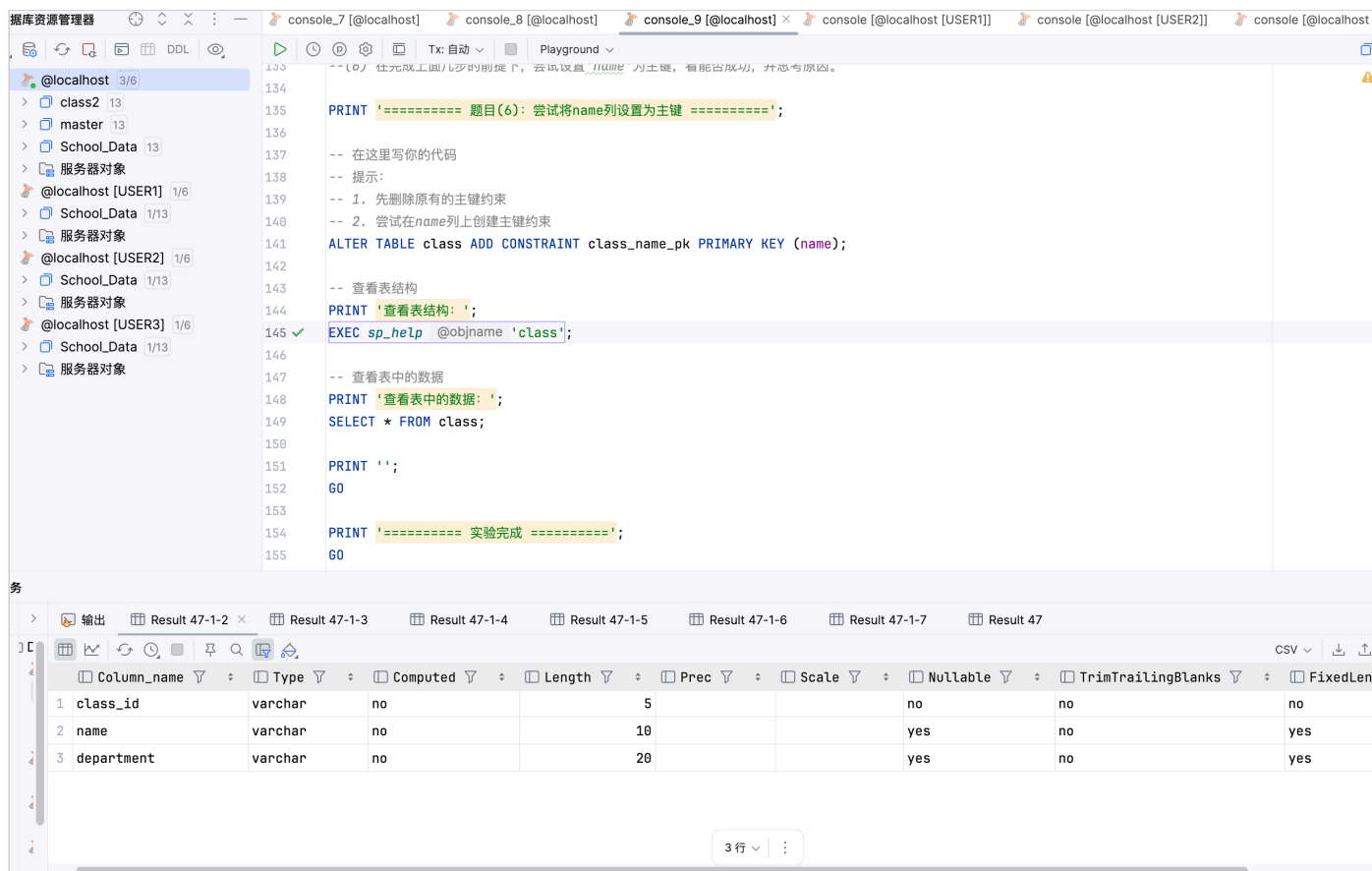


图13: 查看表结构，主键仍为class\_id

终端输出:

```
===== 题目(6): 尝试将name列设置为主键 =====
[2025-11-06 11:02:21] [S0000][1779] 行 1: Table 'class' already has a primary key defined on it.
[2025-11-06 11:02:21] [S0000][1750] 行 1: Could not create constraint or index. See previous errors.
```

### 实验结果分析:

- 报错信息: Table 'class' already has a primary key defined on it.
- **原因1:** 一个表只能有一个主键约束。class表已经在class\_id列上定义了主键，必须先删除原有主键约束，才能在其他列上创建新的主键。
- **原因2:** 即使删除了原有主键，在name列上创建主键也会失败，因为表中存在name列值重复的记录 ('01CSC'出现了2次)。主键要求列值唯一，所以会失败。

### 正确的做法:

1. 先查找主键约束的名称: `SELECT name FROM sys.key_constraints WHERE type = 'PK' AND parent_object_id = OBJECT_ID('class');`
2. 删除原有主键: `ALTER TABLE class DROP CONSTRAINT [约束名];`
3. 确保name列没有重复值
4. 创建新主键: `ALTER TABLE class ADD CONSTRAINT class_name_pk PRIMARY KEY (name);`

## 四、问题与总结

### 1. 主键约束的本质

通过本次实验，我深入理解了主键约束的本质。主键约束实际上是两个约束的组合：NOT NULL约束和UNIQUE约束。在题目(3)中，当我尝试将主键列设置为NULL时，系统明确报错"column does not allow nulls"，这说明主键列自动具有NOT NULL属性。主键的这两个特性共同保证了实体完整性，确保每一行数据都有唯一标识。

## 2. 事务处理与错误控制

题目(2)和(5)展示了事务处理中的错误控制机制。XACT\_ABORT设置对事务行为有决定性影响。在OFF模式下，单个语句的错误不会导致整个事务回滚，这给了开发者更多的灵活性，但也要求更谨慎的错误处理。在ON模式下，任何运行时错误都会自动回滚整个事务，这更符合事务的原子性原则。在实际应用中，应该根据业务需求选择合适的模式。

## 3. 显式事务与隐式提交

题目(4)揭示了一个重要的概念：在没有显式事务的情况下，每条SQL语句都是一个独立的事务，执行成功后立即提交。这就是为什么第一条INSERT成功后，即使第二条INSERT失败，第一条记录也不会被撤销。这种行为在批量操作时需要特别注意，如果希望多条语句作为一个整体要么全部成功要么全部失败，必须使用显式事务。

## 4. 嵌套事务的特殊性

题目(2)中的嵌套事务实验很有意思。虽然SQL Server支持嵌套事务的语法，但实际上内层事务的COMMIT并不会真正提交数据，只有最外层事务的COMMIT才会真正提交。内层事务的错误是否影响外层事务，取决于XACT\_ABORT的设置。这个机制在复杂的存储过程中很有用，但也需要仔细设计错误处理逻辑。

## 5. 主键约束的唯一性

题目(6)让我认识到，一个表只能有一个主键约束。这是关系数据库理论的基本规则，因为主键是表的唯一标识。如果需要在多个列上保证唯一性，应该使用UNIQUE约束而不是主键。而且，即使删除了原有主键，如果新列的数据不满足唯一性要求，也无法创建主键。

## 6. 实体完整性的重要性

实体完整性是数据库完整性的基础。主键约束通过保证每行数据的唯一标识，使得我们可以准确地定位和操作每一条记录。在实际应用中，选择合适的主键非常重要。主键应该是稳定的、不可变的、简洁的。自增ID是常见的选择，但在某些场景下，业务字段的组合也可以作为主键。

## 7. 错误处理的最佳实践

通过这次实验，我总结了一些错误处理的最佳实践。在事务中，应该使用TRY-CATCH块来捕获和处理错误，而不是依赖XACT\_ABORT的自动回滚。在批量操作时，应该使用显式事务来保证原子性。在设计数据库时，应该合理使用约束来防止无效数据的插入，而不是在应用层进行所有验证。

## 8. 实验收获

这次实验让我对SQL Server的事务处理机制有了更深入的理解。主键约束不仅仅是一个简单的唯一性限制，它涉及到事务处理、错误控制、数据完整性等多个方面。在实际开发中，正确理解和使用这些机制，对于保证数据的一致性和可靠性至关重要。